Hadoop Installation & MapReduce Execution Guide

Apache Hadoop 3.3.6 — Pseudo-Distributed Setup on Ubuntu

Step 1 — Update Ubuntu and Install Java 8

```
sudo apt update
sudo apt install openjdk-8-jdk -y
java -version
```

Step 2 — Install and Configure SSH

```
sudo apt install openssh-server openssh-client -y
sudo systemctl start ssh
sudo systemctl enable ssh
mkdir -p ~/.ssh
chmod 700 ~/.ssh
ssh-keygen -t rsa -P "" -f ~/.ssh/id_rsa
cat ~/.ssh/id_rsa.pub >> ~/.ssh/authorized_keys
chmod 600 ~/.ssh/authorized_keys
ssh localhost
exit
```

Step 3 — Download and Extract Hadoop 3.3.6

```
cd ~
wget https://dlcdn.apache.org/hadoop/common/hadoop-3.3.6/hadoop-3.3.6.tar.gz
tar -xzf hadoop-3.3.6.tar.gz
mv hadoop-3.3.6 hadoop
```

Step 4 — Configure Environment Variables

```
nano ~/.bashrc
```

Paste at the bottom of ~/.bashrc

```
export JAVA_HOME=/usr/lib/jvm/java-8-openjdk-amd64
export HADOOP_HOME=$HOME/hadoop
export HADOOP_INSTALL=$HADOOP_HOME
export HADOOP_MAPRED_HOME=$HADOOP_HOME
export HADOOP_COMMON_HOME=$HADOOP_HOME
export HADOOP_HDFS_HOME=$HADOOP_HOME
export YARN_HOME=$HADOOP_HOME
export HADOOP_COMMON_LIB_NATIVE_DIR=$HADOOP_HOME/lib/native
export PATH=$PATH:$HADOOP_HOME/bin:$HADOOP_HOME/sbin
source ~/.bashrc
hadoop version
```

Step 5 — Configure Hadoop XML Files

```
cd $HADOOP_HOME/etc/hadoop
sed -i 's|^#*export JAVA_HOME.*|export JAVA_HOME=/usr/lib/jvm/java-8-openjdk-amd64|' hadoop-env.sh
```

core-site.xml

```
cat > core-site.xml <<'EOF'
<configuration>
    <property>
        <name>fs.defaultFS</name>
        <value>hdfs://localhost:9000</value>
    </property>
</configuration>
EOF
```

hdfs-site.xml

```
cat > hdfs-site.xml <<'EOF'
<configuration>
    <property>
        <name>dfs.replication</name>
```

```
        <value>1</value>
    </property>
</configuration>
EOF
```

mapred-site.xml

```
cp mapred-site.xml.template mapred-site.xml
cat > mapred-site.xml <<'EOF'
<configuration>
    <property>
        <name>mapreduce.framework.name</name>
        <value>yarn</value>
    </property>
</configuration>
EOF
```

yarn-site.xml

```
cat > yarn-site.xml <<'EOF'
<configuration>
    <property>
        <name>yarn.nodemanager.aux-services</name>
        <value>mapreduce_shuffle</value>
    </property>
</configuration>
EOF
```

Step 6 — Format NameNode and Start Hadoop

```
hdfs namenode -format
start-dfs.sh
start-yarn.sh
jps
```

Expected jps output

```
NameNode
DataNode
SecondaryNameNode
ResourceManager
NodeManager
```

Step 7 — Create WordCount Project

```
cd ~
mkdir -p wordcount
cd wordcount
nano WordCount.java
```

WordCount.java — use this code

```
import java.io.IOException;
import java.util.StringTokenizer;

import org.apache.hadoop.conf.Configuration;
import org.apache.hadoop.fs.Path;
import org.apache.hadoop.io.IntWritable;
import org.apache.hadoop.io.Text;
import org.apache.hadoop.mapreduce.Job;
import org.apache.hadoop.mapreduce.Mapper;
import org.apache.hadoop.mapreduce.Reducer;
import org.apache.hadoop.mapreduce.lib.input.FileInputFormat;
import org.apache.hadoop.mapreduce.lib.output.FileOutputFormat;

public class WordCount {

    public static class TokenizerMapper
            extends Mapper<Object, Text, Text, IntWritable> {

        private final static IntWritable one = new IntWritable(1);
        private Text word = new Text();
```

```
        public void map(Object key, Text value, Context context)
                throws IOException, InterruptedException {

            StringTokenizer itr = new StringTokenizer(value.toString());

            while (itr.hasMoreTokens()) {
                word.set(itr.nextToken());
                context.write(word, one);
            }
        }
    }

    public static class IntSumReducer
            extends Reducer<Text, IntWritable, Text, IntWritable> {

        private IntWritable result = new IntWritable();

        public void reduce(Text key, Iterable<IntWritable> values,
                Context context)
                throws IOException, InterruptedException {

            int sum = 0;

            for (IntWritable val : values) {
                sum += val.get();
            }

            result.set(sum);
            context.write(key, result);
        }
    }

    public static void main(String[] args) throws Exception {

        Configuration conf = new Configuration();

        Job job = Job.getInstance(conf, "word count");

        job.setJarByClass(WordCount.class);
        job.setMapperClass(TokenizerMapper.class);
        job.setCombinerClass(IntSumReducer.class);
        job.setReducerClass(IntSumReducer.class);
        job.setOutputKeyClass(Text.class);
        job.setOutputValueClass(IntWritable.class);

        FileInputFormat.addInputPath(job, new Path(args[0]));
        FileOutputFormat.setOutputPath(job, new Path(args[1]));

        System.exit(job.waitForCompletion(true) ? 0 : 1);
    }
}
```

Step 8 — Compile and Create JAR

```
javac -classpath "$(hadoop classpath)" -d . WordCount.java
jar -cvf wordcount.jar WordCount*.class
```

Step 9 — Create Input File

```
echo "hello hadoop hadoop mapreduce" > input.txt
cat input.txt
```

Step 10 — Upload Input to HDFS

```
hdfs dfs -mkdir -p /user/$USER/input
hdfs dfs -put -f input.txt /user/$USER/input/
hdfs dfs -ls /user/$USER/input
```

Step 11 — Run MapReduce

```
hdfs dfs -rm -r -f /user/$USER/output
hadoop jar wordcount.jar WordCount /user/$USER/input /user/$USER/output
```

Step 12 — Display Final Output

```
hdfs dfs -cat /user/$USER/output/part-r-00000
```

Expected Output

```
hadoop  2
hello   1
mapreduce  1
```

Troubleshooting

```
If output already exists:
hdfs dfs -rm -r -f /user/$USER/output

If Hadoop services are not running:
jps
start-dfs.sh
start-yarn.sh

Important: Do NOT run hdfs namenode -format again once your HDFS is working; formatting resets the HDFS metadata.
```